

Software Dependability

AI-enabled Java Microbenchmark Harness (AMBER)

What are we going to do today?

Write efficient microbenchmarks with AMBER

<https://github.com/AntonioTrovato/AMBER>

Useful Links

AI Method Paper

<https://dl.acm.org/doi/pdf/10.1145/3691620.3695017>

AMBER Paper

https://dardin88.github.io/preprints/RECHARGE_2.pdf

Why do Microbenchmarks need Warm-up Iterations

A **microbenchmark** repeatedly execute a small code portion while gathering measurements of its execution time

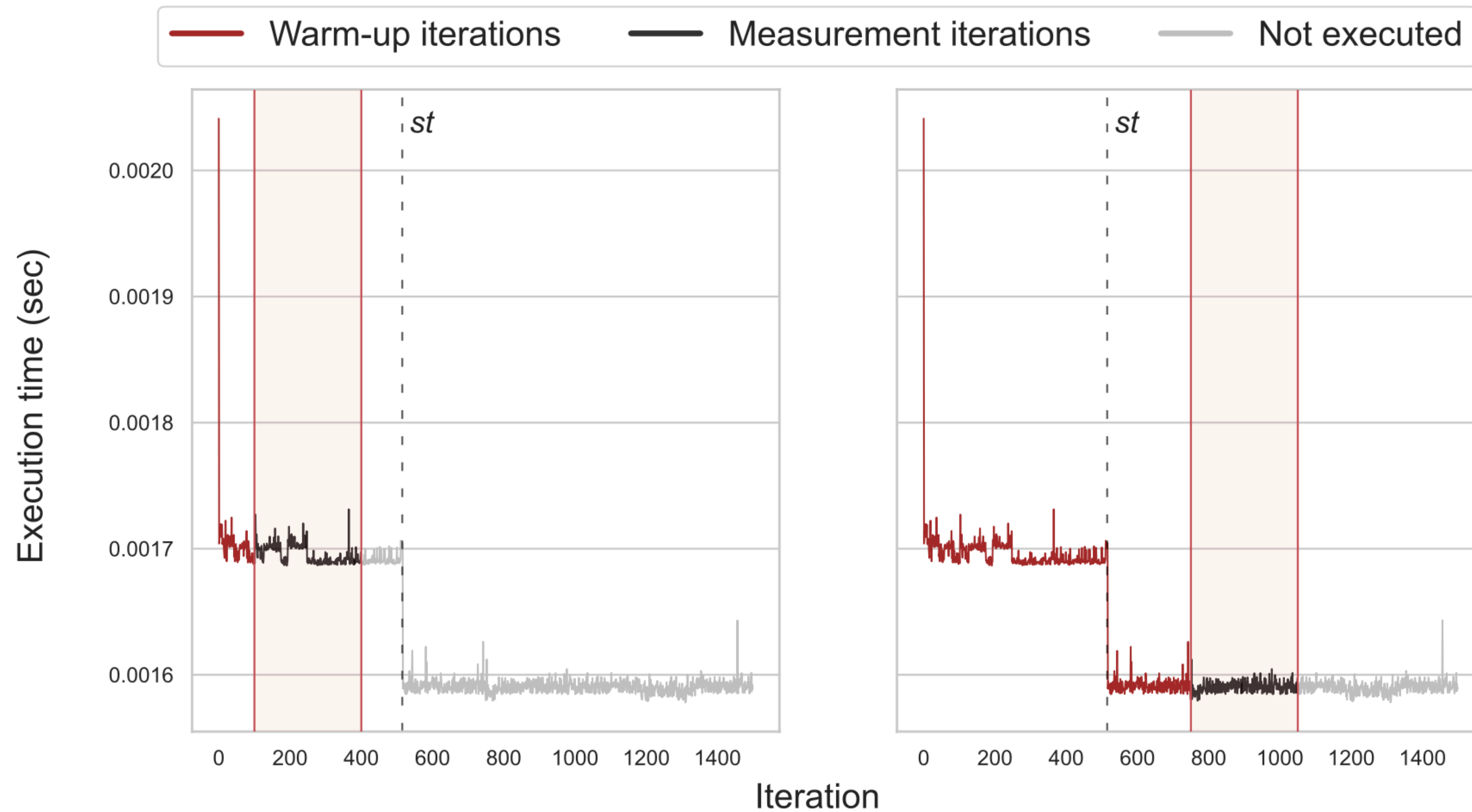
Just-in-time compilation leads to microbenchmarks **performance variability** at the first phase of their execution (**warm-up phase**)

The **JVM** detects **frequently executed loops** or **methods**, **dynamically compiling** them into **optimized machine code**

Software engineers employ **warm-up iteration** to make the **microbenchmarks** reach a **steady-state of performance**

Steady-state: a state in which the JVM optimizations are completed and the code performance stable

The Impact of Warm-up Iterations



A correct estimate of the number of warm-up iterations would lead to **accurate results** and to **sped up performance testing**

Warm-up Iterations Setup: State-of-practice

```
@Fork(1)
public class Basics {

    @Benchmark
    @Warmup(iterations = 5)
    @Measurement(iterations = 5)
    public void measureThroughput() throws Exception {
        TimeUnit.MILLISECONDS.sleep(100);
    }

    @Benchmark
    @Warmup(iterations = 5)
    @Measurement(iterations = 5)
    public void measureAvgTime() throws Exception {
        TimeUnit.MILLISECONDS.sleep(100);
    }
}
```

```
@Warmup(iterations = 500)
@Measurement(iterations = 100)
public class FlattenRangePerf {

    ...

    @Benchmark
    public void observable(Blackhole bh) {
        observable.subscribe(new PerfConsumer(bh));
    }

}
```

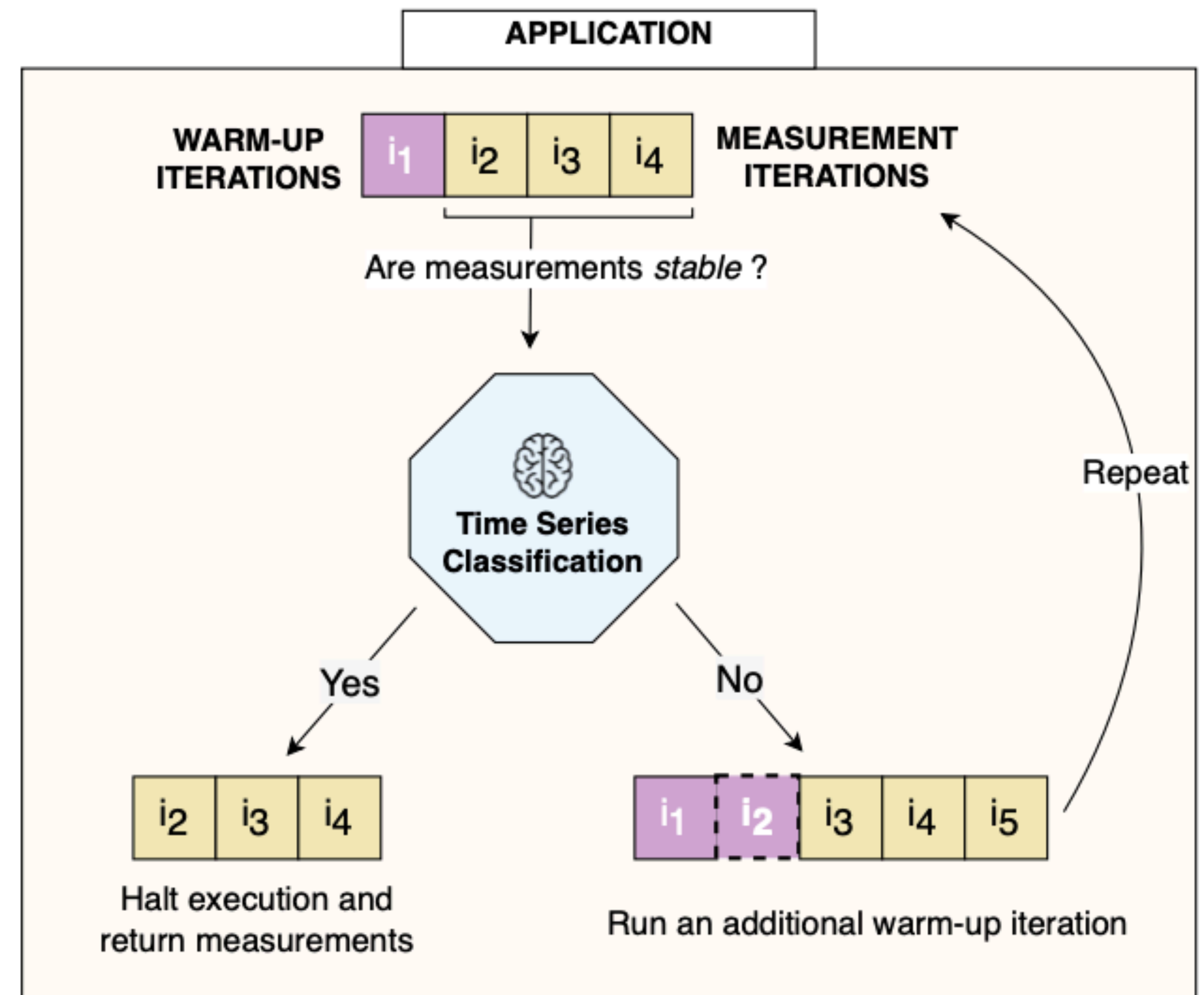
Practitioners usually predefine a **fixed** number of **warm-up iterations** based on their **domain expertise**

JMH allows practitioners to **configure** the warm-up iterations **directly in the microbenchmark's source code**

Problem: prior work has shown that using a fixed configuration easily leads to **suboptimal estimates**

The AMBER Approach

- **AMBER**: a **JMH** extension leveraging **pre-trained Time Series Classification models** to predict whether a of a sequence of measurements is **stable**
- After each iteration, AMBER sends to the TSC model the **last 100 iteration results**
- If the **current window** contains **stable measurements**, the microbenchmark is stopped
- Otherwise, AMBER executes another iteration **until** a **limit** of 500 iterations
- The TSC models are hosted on a **REST API**, so they can be **replaced** with other models exposing the same API



The DynamicHalt Annotation

```
public class FlattenRangePerf {  
    ...  
    @Benchmark  
    @DynamicHalt(model = "OSCNN")  
    public void observable(Blackhole bh) {  
        observable.subscribe(new PerfConsumer(bh));  
    }  
}
```

```
@DynamicHalt(model = "OSCNN")  
public class FlattenRangePerf {  
    ...  
    @Benchmark  
    public void observable(Blackhole bh) {  
        observable.subscribe(new PerfConsumer(bh));  
    }  
}
```

- The **model** parameter indicates the model name
- By **default**, the **host** is set to “localhost”, and **port** to “5001”, but **they can be both configured as model**